Programozási nyelvek 1

Szathmáry László
Debreceni Egyetem
Informatikai Kar

6. előadás

- random számok, nevesített konstansok
- tömb / sztring visszaadása függvényből

(utolsó módosítás: 2021. febr. 6.)

Random számok

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("%d\n", rand());

    return 0;
}
```

Random számok

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    srand(42);

    for (int i = 0; i < 3; ++i)
    {
        printf("%d\t", rand());
    }
    puts("");

    return 0;
}
```

seed érték

Ezzel inicializáljuk a véletlenszám-generátort.

Random számok

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    srand(time(NULL));

    for (int i = 0; i < 3; ++i)
    {
        printf("%d\t", rand());
    }
    puts("");

    return 0;
}
```

Minden indítás során más és más értékkel inicializáljuk a véletlenszám-generátort.

Következmény:

minden indítás során más és más véletlenszámokat kapunk

```c
srand(42);        // inicializálás valamilyen konstans értékkel
                  // Mire jó? Egy kísérletet megismételhetővé tesz.
```

Random számok

Véletlenszámok esetén gyakori igény, hogy egy adott intervallumból szeretnénk kapni egy random számot.

Pl.: generáljunk egy véletlen értéket a [70, 72] zárt intervallumból.

Fogalmazzunk precízen!

- Generáljunk egy random számot 20 és 30 között.
- Generáljunk egy random számot 20 és 30 között (a 30 is benne lehet).
- Generáljunk egy random számot a [20, 30] zárt intervallumon.
- Generate a random number between 20 and 30.
- Generate a random number between 20 and 30 (excl.).
- Generate a random number between 20 and 30 (incl.).

Házi feladat:

A) generáljunk ötöslottó számokat

B) *finomítás*: a generált öt szám legyen **különböző**

Random számok

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/*
    Generáljunk egy véletlenszámot egy [also, felso]
    zárt intervallumon.
*/

int randint(int also, int felso)                // also: 70, felso: 72
{
    int veletlen = rand();                      // pl. 3568
    int intervallum = felso - also + 1;         // 72 - 70 + 1 = 3

    veletlen = veletlen % intervallum;          // 3568 % 3 = 1
    veletlen = also + veletlen;                 // 70 + 1 = 71

    return veletlen;                            // 71
}

int main()
{
    srand(time(NULL));

    printf("%d\n", randint(70, 72));

    return 0;
}
```

Nevesített konstansok

A nevesített konstans olyan programozási eszköz, amelynek három komponense van:

1. név
2. típus
3. érték

Nevesített konstansok

```
#define FENYSEBESSEG_M_S 299792458

int main()
{
    int eredmeny = 2 * FENYSEBESSEG_M_S;

    return 0;
}
```

Nevesített konstansok

```
#define FENYSEBESSEG_M_S 299792458

int main()
{
    int eredmeny = 2 * FENYSEBESSEG_M_S;

    return 0;
}
```

Nevesített konstansok

```
int main()
{
    int eredmeny = 2 * 299792458;

    return 0;
}
```

Az előfeldolgozó által produkált kimenet (módosított C forráskód) megtekintése:

```
$ gcc -E input.c
```

Nevesített konstansok

Egy másik módszer:

```
#include <stdio.h>

const int FENYSEBESSEG_M_S = 299792458;

int main()
{
    int eredmeny = 2 * FENYSEBESSEG_M_S;
    printf("%d\n", eredmeny);

    return 0;
}
```

Tömb visszaadása függvényből

1. Egy függvényből nem tudunk visszaadni egy *lokális* tömböt, mivel a függvény visszatérésekor felszabadul a tömb által lefoglalt memóriaterület.

2. Használhatnánk dinamikus memóriafoglalást, de erről később lesz szó.

3. **Jelenlegi megoldás:** a tömböt a hívó oldalon deklarálom, s átadom egy másik fv.-nek (vagy eljárásnak), ami közvetlenül módosítja a tömböt. A meghívott fél befejeződése után látni fogja a hívó fél a módosításokat.

Tömb visszaadása függvényből

```c
#include <stdio.h>

void array_10(int n, int tomb[])
{
    for (int i = 0; i < n; ++i)
    {
        tomb[i] = i + 1;
    }
}

int main()
{
    // Kell nekem egy tömb, ami egészeket tartalmaz 1-től 10-ig (a 10-et is).

    int szamok[10];
    int meret = 10;

    array_10(meret, szamok);

    for (int i = 0; i < meret; ++i)
    {
        printf("%d\t", szamok[i]);
    }
    puts("");

    return 0;
}
```

Sztring visszaadása függvényből

```c
#include <stdio.h>

#define MAX 100

/*
    Írjunk függvényt, ami visszaadja a "Bea" sztringet.
*/

void insert_name(int n, char s[])
{
    s[0] = 'B';
    s[1] = 'e';
    s[2] = 'a';
    s[3] = '\0';
}

int main()
{
    char szoveg[MAX];
    int meret = MAX;

    insert_name(meret, szoveg);

    printf("%s\n", szoveg);

    return 0;
}
```

manuális módszer

Sztring visszaadása függvényből

```c
#include <stdio.h>
#include <string.h>

#define MAX 100

/*
    Írjunk függvényt, ami visszaadja a "Bea" sztringet.
*/

void insert_name(int n, char s[])
{
    strcpy(s, "Bea");
}

int main()
{
    char szoveg[MAX];
    int meret = MAX;

    insert_name(meret, szoveg);

    printf("%s\n", szoveg);

    return 0;
}
```

Házi feladat

- A K & R-féle „C Bibliában” nézzék át azokat a részeket, amikről szó volt az előadáson.
- Juhász István jegyzetéből nézzék át azokat a fogalmakat, amikről szó volt az előadáson (link).